

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 09 · LECTURE NOTES

Asymptotics

What CSCE 629 is actually about.

MODULE OUTCOMES

- State the definitions of O , Ω , and Θ precisely.
- Prove an asymptotic bound from the definition.
- Compare growth rates reliably.
- Explain what asymptotic analysis hides.
- Use the notation without the standard abuses.

1 The definitions

$f(n) = O(g(n))$: there exist $c > 0$ and n_0 with $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

- **An upper bound, eventually, up to a constant factor** — and the c and the n_0 are exactly what you must exhibit in order to prove the claim (§2).
- **$f(n) = \Omega(g(n))$ is the same definition with the inequality reversed** — a lower bound — **used far less often than it should be**, and it is **what a hardness result actually proves** (CSCE 637's lower bounds are all Ω statements).
- **$f(n) = \Theta(g(n))$ means both at once**, with possibly different constants — **a tight bound** — and **this is usually what you mean when you say 'the algorithm is $O(n \log n)$ '**, so saying O is strictly weaker than what you intended.
- **Exhibit the constant and the threshold: that is what proving an asymptotic bound means**, and intuition about which function grows faster is not a proof. Project 2 requires asymptotic proofs that exhibit c and n_0 explicitly, for exactly this reason.

AND THE NOTATION IS AN ABUSE THAT EVERYBODY COMMITS, WHICH IS WORTH UNDERSTANDING ONCE

$O(g)$ is properly a *set of functions*, and ' $f = O(g)$ ' really means ' $f \in O(g)$ ' — which is why the relation is not symmetric and why you cannot cancel terms across it.

So ' $O(n) = O(n^2)$ ' is a true statement read left to right and a false one read right to left — which is the clearest available sign that the equals sign is doing something unusual here, since equality is symmetric and this is not.

And an expression like $n^2 + O(n)$ means ' n^2 plus some function that is in $O(n)$ ' — which is genuinely useful shorthand, and is the reason the abuse has survived a century of complaints about it.

So read the sign as 'is' rather than 'equals' — and never manipulate an O -expression as though it were an equation, which is where the standard errors come from (subtracting $O(n)$ from both sides, for instance, is meaningless).

2 Proving a bound

CLAIM: $3n^2 + 7n + 2 = O(n^2)$.

PROOF. Take $c = 12$ and $n_0 = 1$.

For $n \geq 1$ we have $n \leq n^2$ and $1 \leq n^2$, so

$$3n^2 + 7n + 2 \leq 3n^2 + 7n^2 + 2n^2 = 12n^2.$$

Hence $f(n) \leq 12n^2$ for all $n \geq 1$. []

THAT IS THE WHOLE TECHNIQUE

bound every lower-order term by the leading one
add up the coefficients
and that sum is your c

FOR A LOWER BOUND

CLAIM: $3n^2 + 7n + 2 = \Omega(n^2)$.

Take $c = 3$ and $n_0 = 1$: the remaining terms are positive, so $f(n) \geq 3n^2$ immediately. []

AND THEREFORE $\Theta(n^2)$, by both together.

Two short proofs, and the tight bound follows.

Bound every lower-order term by the leading one, add the coefficients, and that sum is your constant — which is the whole technique for polynomials, and it adapts directly to anything else you can bound termwise. **Lower bounds are usually easier than upper bounds and are usually omitted**, which is a shame: the Ω proof above is two lines, and quoting Θ instead of O is a strictly stronger and more useful claim.

3 Comparing growth

Growth	Typical example	At $n = 1,000,000$
$O(1)$	Hash lookup, array index.	1
$O(\log n)$	Binary search, balanced tree lookup.	About 20.
$O(n)$	Linear scan, counting.	10^6
$O(n \log n)$	Comparison sorting, FFT.	About 2×10^7 — still entirely fine.
$O(n^2)$	Naive all-pairs comparison.	10^{12} — already too slow, by a lot.
$O(2^n)$	Enumerating all subsets.	Unimaginable — and this is why CSCE 637 exists.

The gap between $n \log n$ and n^2 is where most algorithmic work happens — the others are usually decided by the problem rather than by cleverness (you cannot sort in $O(n)$ by comparisons, and you cannot search an unsorted array in less than $O(n)$). **Knowing these numbers by feel is what makes a complexity claim meaningful** rather than decorative: 'quadratic' should immediately read as 'not at a million'.

4 What it hides

- The constant factor — an $O(n)$ algorithm with a constant of 10,000 loses to an $O(n \log n)$ one with a constant of 2 for every input size you will ever actually run, and the asymptotic comparison says the opposite.
- The threshold n_0 — 'eventually' may begin past every input size that will ever exist, which is the **galactic-algorithm problem**: asymptotically superior methods that are never worth running.
- The memory hierarchy — two $O(n)$ algorithms can differ by a factor of fifty on cache behaviour alone, which is CSCE 614's entire subject and is invisible to this notation.
- The distinction between worst, average, and amortised case — O is worst case unless stated otherwise, and quicksort is the standard example of the worst case being the wrong question (quadratic worst case, $n \log n$ expected, and it wins in practice).
- So asymptotics tells you which algorithm to reach for first, and measurement tells you which one is actually faster — and both are needed; treating either as the whole answer is the common error, in opposite directions.

AND WHY THE NOTATION EXISTS AT ALL

Because the exact step count depends on the machine, the compiler, the libraries, and the day — and the growth rate does not. A claim about growth survives a hardware generation; a claim about milliseconds does not.

So asymptotic analysis is an abstraction that discards precisely the machine-dependent part — which is what makes an algorithmic claim portable and durable, and is why the field can have results rather than only benchmarks.

And the cost of that abstraction is §4: everything it discarded is sometimes the thing that matters, and knowing when is experience rather than theory.

Which is CSCE 629 and CSCE 614 in one sentence — one course gives you the growth rate and the other gives you the constant — and the program puts them in the same semester for exactly that reason.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CLRS, chapter 3 (library copy)	§§1 and 2 — the definitions in the form CSCE 629 uses, with the set-membership point made explicitly.
Lehman, Leighton & Meyer, chapter 14 (free)	§§1 to 3, free — asymptotics with the common abuses catalogued.
Knuth — Big Omicron and big Omega and big Theta (free)	§1's callout — two pages on the notation and why the equals sign is what it is, by the person who standardised it.
Sedgewick & Wayne, the analysis lectures (free)	§4 — measurement alongside analysis, which is the practice this module argues for.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State all three definitions** from memory.
- 2 **Prove three polynomial bounds**, exhibiting c and n_0 .
- 3 **Prove the matching lower bounds**, and conclude Θ .
- 4 **Explain why $O(n) = O(n^2)$** is true one way only.

- 5 **Find an invalid O-manipulation** in the wild, or construct one.
- 6 **Order ten functions** by growth and prove three of the comparisons.
- 7 **Compute the hierarchy table** for $n = 10^9$.
- 8 **Find two algorithms with the same O** and measure a 10x difference.
- 9 **Find an asymptotically better algorithm** that loses in practice, and say why.
- 10 **State what worst, average, and amortised each mean**, with an example.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the definitions of O , Ω , and Θ .
- 2 What must you exhibit to prove an O bound?
- 3 Why is ' $f = O(g)$ ' an abuse, and what does it really say?
- 4 Why can you not manipulate O -expressions as equations?
- 5 Give the technique for proving a polynomial bound.
- 6 Why is the lower bound usually easier?
- 7 Give the growth hierarchy and the values at a million.
- 8 Where does most algorithmic work happen?
- 9 Name four things asymptotics hides.
- 10 Why does the notation exist, and what does it cost?